

Terraform and AWS Infrastructure for Twenty CRM

1. Objective

Provision AWS infrastructure consisting of an EC2 instance and an Amazon ECR repository using Terraform, while reusing the account's existing default VPC. No new VPC is created. The Twenty CRM Docker image is pushed to ECR, and the EC2 instance automatically authenticates to ECR, pulls the image, and runs the application through a boot-time user-data script with retry logic.

2. File-by-File Explanation

provider.tf

Declares the required Terraform version ($\geq 1.5.0$) and AWS provider (~ 5.0), pinned to us-east-1. The configuration uses local Terraform state because this is a single-person, short-lived deployment. `default_tags` automatically applies Project, ManagedBy, and Environment tags to resources.

variables.tf

Centralizes every configurable deployment value:

- Networking: `use_default_vpc`, `existing_vpc_id`, `existing_subnet_id`. These allow the configuration to auto-discover the default VPC/subnet or target a specific one.
- EC2: `instance_type`, `key_pair_name`, `ssh_ingress_cidr`, `app_port`, `root_volume_size`, `ami_id`.
- ECR: `ecr_repository_name`, `ecr_image_tag`, `image_mutability`.
- IAM: `iam_instance_profile_name`. This references the pre-existing EC2ECRPullRole instance profile rather than creating a new IAM role because the AWS IAM user lacks `iam:CreateRole` permission.

main.tf

- Networking data sources: `data.aws_vpcs` (plural) is used to look up the default VPC instead of `data.aws_vpc` (singular). The singular data source also calls `ec2:DescribeVpcAttribute` for DNS settings, which this IAM user is not authorized to perform. `data.aws_subnets` then finds subnets inside the selected VPC. No `aws_vpc` resource is created.
- AMI lookup: `data.aws_ami` dynamically retrieves the latest Canonical Ubuntu 20.04 AMI instead of relying on a hardcoded AMI ID that could become outdated.
- ECR repository: `aws_ecr_repository.twenty_crm` creates a private repository with `scan_on_push = true` for basic vulnerability scanning and `force_delete = true` so terraform destroy can remove the repository after an image has been pushed.
- Security group: `aws_security_group.twenty_crm_sg` allows SSH on port 22 only from `var.ssh_ingress_cidr` and exposes the application port 2020 to the internet so the CRM UI can be reached. Outbound traffic is open.
- EC2 instance: `aws_instance.twenty_crm` launches into the discovered default subnet, attaches the security group and pre-existing IAM instance profile, and renders `templates/user_data.sh.tpl` with the ECR repository URL, image tag, AWS region, and application port.
- `user_data_replace_on_change = true` causes the instance to be recreated when the user-data script changes. This mechanism was used when the application port was corrected from 3000 to 2020.

templates/user_data.sh.tpl

The script runs automatically when the EC2 instance boots:

1. Installs Docker and starts the Docker service.
2. Authenticates to Amazon ECR using `aws ecr get-login-password` and the EC2 instance's IAM instance profile. No AWS credentials are hardcoded in the script.
3. Loops and retries `docker pull <ecr_repo_url>:<tag>` every 30 seconds until the image becomes available. This allows the instance to boot before the image is pushed and recover automatically.
4. Runs the application container with `docker run -d --name twenty-crm -p 2020:2020 --restart unless-stopped <ecr_repo_url>:<tag>`.

outputs.tf

Exports the information needed to interact with the deployment, including the ECR repository URL and ARN, EC2 instance ID, public IP, public DNS name, computed application URL, and the VPC and subnet IDs used.

terraform.tfvars

Stores the actual deployment values for this run, including `project_name = "twenty-crm-fathima"`, `app_port = 2020`, the personal EC2 key pair name, and the personal IP address used for SSH ingress. This file is gitignored and should not be committed to the repository.

3. Step-by-Step Deployment Process

Step 1 — Provision Infrastructure

The following Terraform commands were executed:

```
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
```

Final Terraform apply result:

```
Apply complete! Resources: 1 added, 2 changed, 1 destroyed.

Outputs:
app_url          = "http://100.53.60.131:2020"
ec2_instance_id  = "i-0a8ea7578a672904f"
ec2_public_dns   = "ec2-100-53-60-131.compute-1.amazonaws.com"
ec2_public_ip    = "100.53.60.131"
ecr_repository_arn = "arn:aws:ecr:us-east-1:579138738751:repository/twenty-crm"
ecr_repository_url = "579138738751.dkr.ecr.us-east-1.amazonaws.com/twenty-crm"
subnet_id        = "subnet-078d52bfe579c74f2"
vpc_id           = "vpc-0c241509159132524"
```

The one destroyed resource reflects the earlier EC2 instance being replaced after the user-data script was corrected. The replacement ensured the updated application port and boot-time configuration were applied.

```

aws_instance.twenty_crm: Still destroying... [id=i-0cc31292e32a02c0a, 00m20s elapsed]
aws_instance.twenty_crm: Still destroying... [id=i-0cc31292e32a02c0a, 00m30s elapsed]
aws_instance.twenty_crm: Destruction complete after 33s
aws_ecr_repository.twenty_crm: Modifying... [id=twenty-crm]
aws_security_group.twenty_crm_sg: Modifying... [id=sg-0ef6dc5db09f3545b]
aws_ecr_repository.twenty_crm: Modifications complete after 3s [id=twenty-crm]
aws_security_group.twenty_crm_sg: Modifications complete after 3s [id=sg-0ef6dc5db09f3545b]
aws_instance.twenty_crm: Creating...
aws_instance.twenty_crm: Still creating... [00m10s elapsed]
aws_instance.twenty_crm: Creation complete after 16s [id=i-0a8ea7578a672904f]

```

Apply complete! Resources: 1 added, 2 changed, 1 destroyed.

Outputs:

```

app_url = "http://100.53.60.131:2020"
ec2_instance_id = "i-0a8ea7578a672904f"
ec2_public_dns = "ec2-100-53-60-131.compute-1.amazonaws.com"
ec2_public_ip = "100.53.60.131"
ecr_repository_arn = "arn:aws:ecr:us-east-1:579138738751:repository/twenty-crm"
ecr_repository_url = "579138738751.dkr.ecr.us-east-1.amazonaws.com/twenty-crm"
subnet_id = "subnet-078d52bfe579c74f2"
vpc_id = "vpc-0c241509159132524"
PS C:\Users\Public\Devops\terraform-twenty-crm\terraform-twenty-crm> docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
579138738751.dkr.ecr.us-east-1.amazonaws.com/twenty-crm:latest	53381e68f6fa	1.84GB	271MB	U
devops-crm-project-app:latest	399277ba6a7f	1.29GB	218MB	U
twentycrm/twenty-app-dev:latest	53381e68f6fa	1.84GB	271MB	U

PS C:\Users\Public\Devops\terraform-twenty-crm\terraform-twenty-crm> ^C
PS C:\Users\Public\Devops\terraform-twenty-crm\terraform-twenty-crm> |



Step 2 — Identify the Correct Docker Image

Two local Docker images were available:

- devops-crm-project-app:latest — a custom build from the repository's Dockerfile. It is a development/synchronization helper image, not the actual CRM application.
- twentycrm/twenty-app-dev:latest — the official all-in-one Twenty CRM image. It bundles the server and its own PostgreSQL database and serves the application on port 2020.

The second image was confirmed as the correct image to deploy.

Step 3 — Tag and Push the Image to ECR

The Twenty CRM image was tagged with the ECR repository URL:

```
docker tag twentycrm/twenty-app-dev:latest 579138738751.dkr.ecr.us-east-1.amazonaws.com/twenty-crm:latest
```

ECR authentication was performed with:

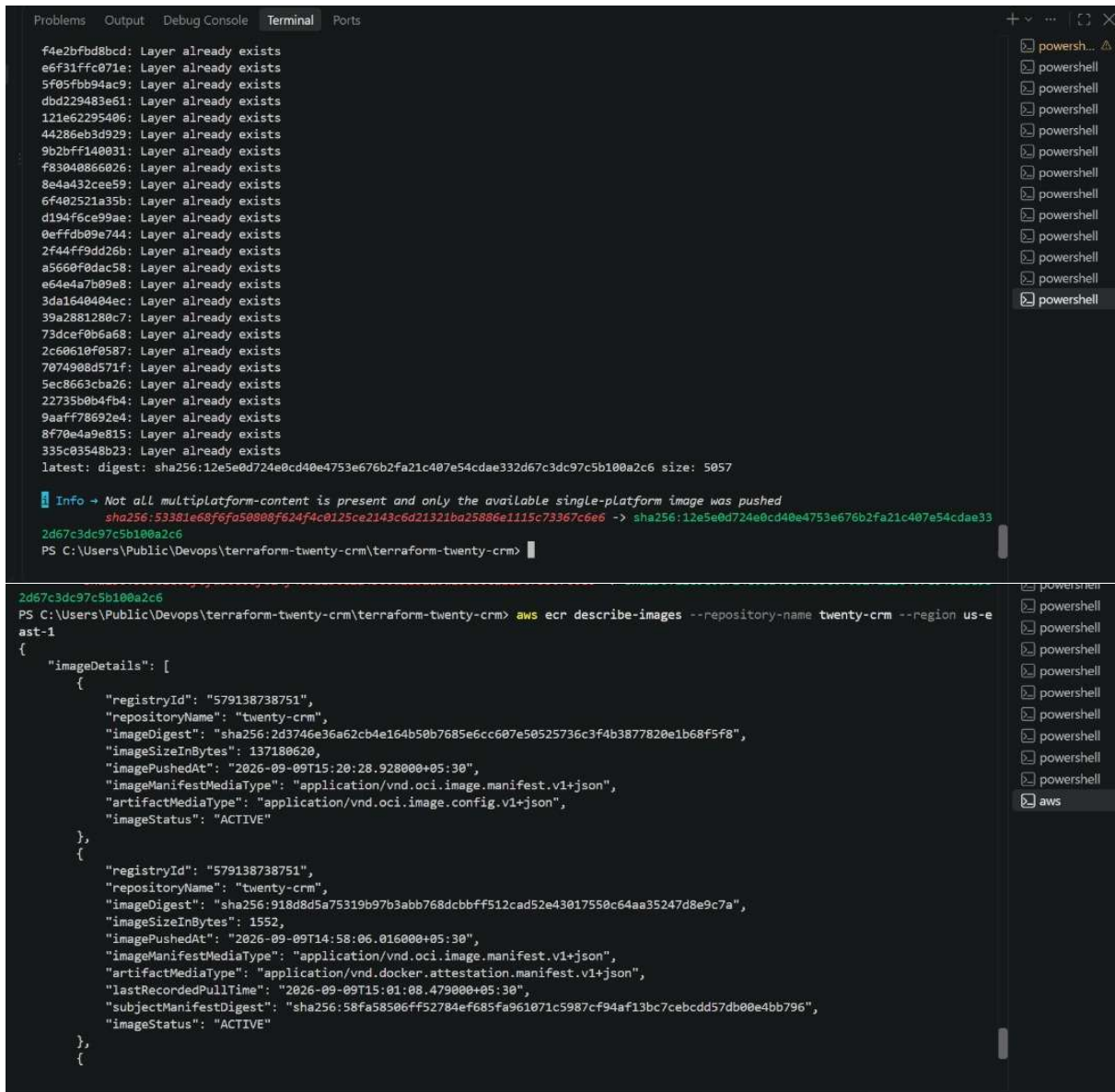
```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 579138738751.dkr.ecr.us-east-1.amazonaws.com
```

The image was then pushed:

```
docker push 579138738751.dkr.ecr.us-east-1.amazonaws.com/twenty-crm:latest
```

The local tag was verified to point to the same image ID as the source image, 53381e68f6fa, confirming a clean retag rather than a rebuild. The push was also verified in ECR with:

```
aws ecr describe-images --repository-name twenty-crm --region us-east-1
```



The screenshot shows a terminal window with two panels. The top panel displays the output of a Docker build command, showing 33 layers being created, each with the message 'Layer already exists'. The bottom panel shows the output of the command `aws ecr describe-images --repository-name twenty-crm --region us-east-1`, which returns a JSON object containing details for two images in the 'twenty-crm' repository. The right sidebar of the terminal shows a list of open tabs, including multiple 'powershell' tabs and one 'aws' tab.

```
f4e2bfbd8bcd: Layer already exists
e6f31ffc071e: Layer already exists
5f05fbb94ac9: Layer already exists
dbd229483e61: Layer already exists
121e62295406: Layer already exists
44286eb3d929: Layer already exists
9b2bfff140031: Layer already exists
f83040866026: Layer already exists
8e4a432cee59: Layer already exists
6f402521a35b: Layer already exists
d194f6ce99ae: Layer already exists
0effdb09e744: Layer already exists
2f44ff9dd26b: Layer already exists
a5660f0dac58: Layer already exists
e64e4a7b09e8: Layer already exists
3da1640404ec: Layer already exists
39a2881280c7: Layer already exists
73dcef0b6a68: Layer already exists
2c60610f0587: Layer already exists
7074908d571f: Layer already exists
5ec8663cba26: Layer already exists
22735b0b4fb4: Layer already exists
9aaff78692e4: Layer already exists
8f70e4a9e815: Layer already exists
335c03548b23: Layer already exists
latest: digest: sha256:12e5e0d724e0cd40e4753e676b2fa21c407e54cdae332d67c3dc97c5b100a2c6 size: 5057

Info -> Not all multiplatform-content is present and only the available single-platform image was pushed
sha256:53381e68f6fa50808f624f4c0125ce2143c6d21321ba25806e1115c73967c6e6 -> sha256:12e5e0d724e0cd40e4753e676b2fa21c407e54cdae33
2d67c3dc97c5b100a2c6
PS C:\Users\Public\Devops\terraform-twenty-crm\terraform-twenty-crm>

2d67c3dc97c5b100a2c6
PS C:\Users\Public\Devops\terraform-twenty-crm\terraform-twenty-crm> aws ecr describe-images --repository-name twenty-crm --region us-east-1
{
  "imageDetails": [
    {
      "registryId": "579138738751",
      "repositoryName": "twenty-crm",
      "imageDigest": "sha256:2d3746e36a62cb4e164b50b7685e6cc607e50525736c3f4b3877820e1b68f5f8",
      "imageSizeInBytes": 137180620,
      "imagePushedAt": "2026-09-09T15:20:28.928000+05:30",
      "imageManifestMediaType": "application/vnd.oci.image.manifest.v1+json",
      "artifactMediaType": "application/vnd.oci.image.config.v1+json",
      "imageStatus": "ACTIVE"
    },
    {
      "registryId": "579138738751",
      "repositoryName": "twenty-crm",
      "imageDigest": "sha256:918d8d5a75319b97b3abb768dcbbff512cad52e43017550c64aa35247d8e9c7a",
      "imageSizeInBytes": 1552,
      "imagePushedAt": "2026-09-09T14:58:06.016000+05:30",
      "imageManifestMediaType": "application/vnd.oci.image.manifest.v1+json",
      "artifactMediaType": "application/vnd.docker.attestation.manifest.v1+json",
      "lastRecordedPullTime": "2026-09-09T15:01:08.479000+05:30",
      "subjectManifestDigest": "sha256:58fa58506ff52784ef685fa961071c5987cf94af13bc7cebcdd57db00e4bb796",
      "imageStatus": "ACTIVE"
    }
  ]
}
```

Step 4 — EC2 Boot Automation

During boot, the EC2 user-data script installed Docker, authenticated to ECR using the attached IAM instance profile, and entered the retry loop. Once the image became available in ECR, the script pulled it automatically and started the Twenty CRM container without manual intervention.

The deployment was checked using:

```
ssh -i twenty-crm-key.pem ubuntu@100.53.60.131
docker images
```

```
Problems Output Debug Console Terminal Ports

Usage of /: 29.7% of 18.25GB    Processes: 137
Memory usage: 65%             Users logged in: 0
Swap usage: 0%                IPv4 address for ens5: 172.31.2.74

Expanded Security Maintenance for Applications is not enabled.

148 updates can be applied immediately.
114 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-2-74:~$ sudo cat /var/log/user-data.log
cat: /var/log/user-data.log: No such file or directory
ubuntu@ip-172-31-2-74:~$ docker images

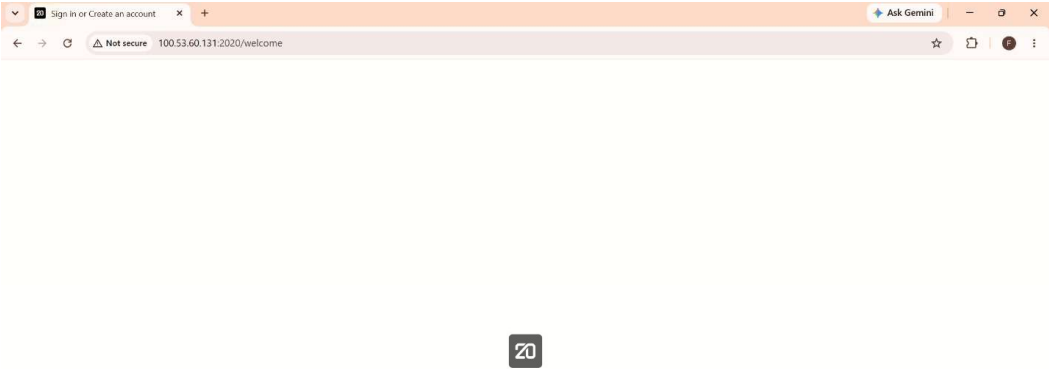
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
579138738751.dkr.ecr.us-east-1.amazonaws.com/twenty-crm:latest 247b98a5f190 1.77GB      254MB        U
postgres:16                    f1c3376c26f2 642MB       166MB        U
redis:7                          71da9275c5f3 178MB       46MB         U
ubuntu@ip-172-31-2-74:~$
```

Step 5 — Verify the Application

The application endpoint was tested with:

```
curl http://100.53.60.131:2020
```

The application was then opened in a browser at `http://100.53.60.131:2020`. The Twenty CRM UI loaded successfully, confirming that the application was running end-to-end on EC2 and that the image had been pulled from the private ECR repository.



4. Cleanup

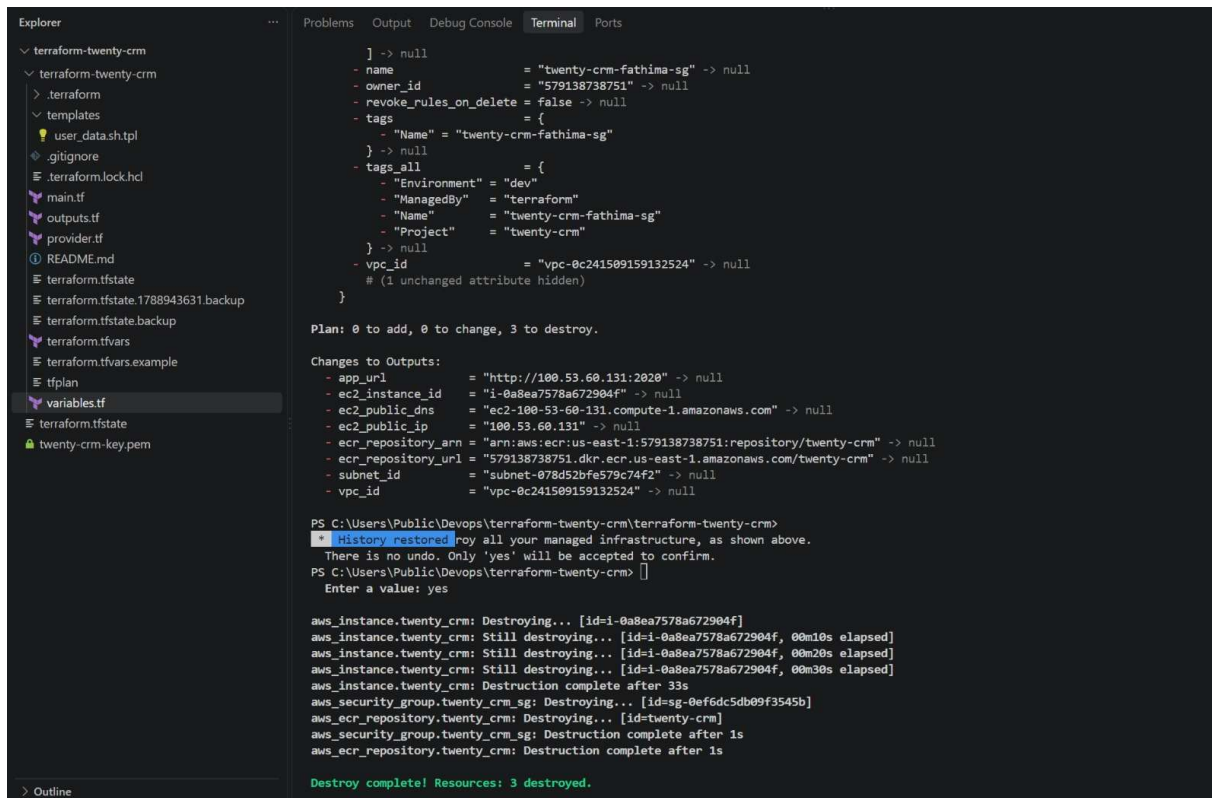
After verification, all Terraform-managed resources were removed using:

```
terraform destroy
```

Cleanup was confirmed with the following AWS CLI commands:

```
aws ec2 describe-instances --instance-ids i-0a8ea7578a672904f --region us-east-1
aws ecr describe-repositories --repository-names twenty-crm --region us-east-1
```

Both commands returned "not found", confirming that the EC2 instance was terminated and the ECR repository, including its pushed image, was removed successfully.



The screenshot shows a VS Code editor with a file explorer on the left and a terminal window on the right. The file explorer shows a project structure for 'terraform-twenty-crm' with files like 'main.tf', 'outputs.tf', 'provider.tf', 'README.md', 'terraform.tfstate', 'terraform.tfstate.backup', 'terraform.tfvars', 'terraform.tfvars.example', 'tfplan', 'variables.tf', and 'twenty-crm-key.pem'. The terminal window shows the output of a Terraform destroy command. It lists the resources to be destroyed: 'aws_instance.twenty_crm', 'aws_security_group.twenty_crm_sg', and 'aws_ecr_repository.twenty_crm'. The output shows the progress of destroying each resource, with the EC2 instance taking 33 seconds and the security group and ECR repository taking 1 second each. The final output is 'Destroy complete! Resources: 3 destroyed.'

```
    ] -> null
    - name                = "twenty-crm-fathima-sg" -> null
    - owner_id            = "579138738751" -> null
    - revoke_rules_on_delete = false -> null
    - tags                = {
      - "Name" = "twenty-crm-fathima-sg"
    } -> null
    - tags_all            = {
      - "Environment" = "dev"
      - "ManagedBy"  = "terraform"
      - "Name"        = "twenty-crm-fathima-sg"
      - "Project"     = "twenty-crm"
    } -> null
    - vpc_id              = "vpc-0c241509159132524" -> null
    # (1 unchanged attribute hidden)
  }

Plan: 0 to add, 0 to change, 3 to destroy.

Changes to Outputs:
  - app_url          = "http://100.53.60.131:2020" -> null
  - ec2_instance_id  = "i-0a8ea7578a672904f" -> null
  - ec2_public_dns   = "ec2-100-53-60-131.compute-1.amazonaws.com" -> null
  - ec2_public_ip    = "100.53.60.131" -> null
  - ecr_repository_arn = "arn:aws:ecr:us-east-1:579138738751:repository/twenty-crm" -> null
  - ecr_repository_url = "579138738751.dkr.ecr.us-east-1.amazonaws.com/twenty-crm" -> null
  - subnet_id        = "subnet-078d52bfe579c74f2" -> null
  - vpc_id            = "vpc-0c241509159132524" -> null

PS C:\Users\Public\Devops\terraform-twenty-crm\terraform-twenty-crm>
* History restored. Destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.
PS C:\Users\Public\Devops\terraform-twenty-crm>
Enter a value: yes

aws_instance.twenty_crm: Destroying... [id=i-0a8ea7578a672904f]
aws_instance.twenty_crm: Still destroying... [id=i-0a8ea7578a672904f, 00m10s elapsed]
aws_instance.twenty_crm: Still destroying... [id=i-0a8ea7578a672904f, 00m20s elapsed]
aws_instance.twenty_crm: Still destroying... [id=i-0a8ea7578a672904f, 00m30s elapsed]
aws_instance.twenty_crm: Destruction complete after 33s
aws_security_group.twenty_crm_sg: Destroying... [id=sg-0ef6dc5db09f3545b]
aws_ecr_repository.twenty_crm: Destroying... [id=twenty-crm]
aws_security_group.twenty_crm_sg: Destruction complete after 1s
aws_ecr_repository.twenty_crm: Destruction complete after 1s

Destroy complete! Resources: 3 destroyed.
```

5. Conclusion

Terraform successfully provisioned the required ECR repository and EC2 instance in the account's existing default VPC without creating a new VPC. The Twenty CRM Docker image was tagged, pushed to the private ECR repository, and automatically pulled and started on EC2 through a self-healing user-data script with retry logic. The application was verified through the browser at <http://100.53.60.131:2020>, and the deployment was fully cleaned up afterward with AWS CLI verification.